

Exercices d'Algorithmique et programmation impérative

Série 2 – Fonctions et instructions conditionnelles — Le jeu de la devinette

(3 octobre 2024)

Département d'Informatique – Faculté des Sciences – UMONS

Pré-requis : expressions booléennes; instructions conditionnelles; fonctions et fonctions booléennes (cours jusqu'au **Chapitre 4**).

Objectifs : être capable d'appeler une fonction; de créer sa propre fonction et son propre module; comprendre la différence entre la valeur de retour d'une fonction et l'affichage; utiliser Python en mode script.

1 Le contrat

1.1 Jeu de la devinette

Afin de mettre en pratique ce que vous avez appris durant le cours théorique, vous allez programmer un jeu simple dont le principe est le suivant : générer un nombre aléatoire compris entre 1 et 100 et le faire deviner à un joueur.

Le joueur dispose de cinq tentatives pour trouver le nombre généré. S'il le devine correctement, un message de félicitations s'affiche. Dans le cas contraire, au bout des cinq tentatives, un message d'échec conclut la partie.

Après chaque tentative ratée, un message d'encouragement doit être affiché. Le format exact du message doit être : "Pas de chance, essayez encore."

Avant chaque tentative, le programme doit demander au joueur d'entrer un nombre en affichant le message exact suivant : "Entrez le nombre qui a été généré aléatoirement :".

1.2 Développer avec des scripts

Jusqu'à présent, vous avez utilisé l'interpréteur Python pour exécuter votre code. Cet outil est très pratique pour tester des bouts de code et se familiariser avec le langage, mais il n'est pas adapté pour développer des programmes plus complexes. Pour cela, vous allez écrire des scripts.

Un script est un fichier texte contenant du code Python. Par convention, les fichiers de script ont l'extension `.py`. Pour exécuter un script, vous devez, dans un terminal, utiliser la commande `python`¹ suivie du nom du fichier de script. Par exemple, si votre script s'appelle `main.py`, vous exécuterez la commande `python main.py`.

1.3 À toi de jouer !

1.3.1 Basique, simple

Pour commencer, vous créez un fichier nommé `devinette.py` dans lequel vous écrirez votre script.

Pour générer un nombre aléatoirement, vous pouvez utiliser la fonction `randint` du module `random`. Pour rappel, vous pouvez consulter la documentation (en ligne ou via la fonction "help") pour savoir comment utiliser une fonction spécifique.

Notez que lorsque l'on récupère l'entrée d'un utilisateur, celle-ci est une chaîne de caractère. Vous pouvez utiliser la méthode² `isdigit()` pour vérifier si une chaîne de caractères ne contient que des chiffres et la fonction `int()` pour convertir une chaîne en entier.

1. La commande sur votre machine peut différer : `python`, `python3`, `python3.11`, ...

2. Vous aborderez la notion de méthode plus tard dans le cours. Pour l'instant, retenez que pour vérifier si une chaîne "14" ne contient que des chiffres, vous devez utiliser : `"14".isdigit()`.

- 1 Commencez par créer un programme de base qui correspond au jeu décrit dans la section 1.1. N'oubliez pas de l'exécuter pour vérifier qu'il fonctionne comme prévu.

1.3.2 De gros indices

Pour l'instant, les chances de trouver le nombre généré sont assez faibles. Pour que ce soit plus intéressant, vous décidez de donner un nouvel indice avant chaque tentative. Le premier indice est donc donné avant la première tentative, le deuxième avant la deuxième tentative, et ainsi de suite jusqu'au dernier indice. Avant chaque indice, affichez à l'écran un message qui dit exactement : "Voici un indice."

- 2 Dans l'ordre, implémentez le nécessaire afin d'afficher les indices (en choisissant l'approprié selon la tentative) indiquant si le nombre :

1. est pair ;
2. est un multiple de 3 ;
3. est inférieur ou strictement supérieur à 50 ;
4. est une puissance de 2 ; et
5. est un carré.

N'oubliez pas de tester votre implémentation, comme d'habitude. Faites vérifier votre implémentation par un assistant (ou élève-assistant) avant de procéder à la suite du contrat.

1.3.3 Comme de chemise

Finalement, vous avez changé d'avis sur plusieurs choses.

- Vous trouvez que le message avant chaque tentative n'est pas assez poli, vous décidez donc d'ajouter ", s'il vous plaît" à la fin de celui-ci.
- Vous trouvez que votre message d'encouragement n'est pas assez encourageant. Vous décidez de lui ajouter du tonique en le formatant en : "Pas de chance!!! Essayez encore!!!!"
- Vous trouvez que votre message pour les indices est un peu trop froid. Vous décidez de le rendre un peu plus joyeux et d'afficher à la place : "Pour vous aider dans votre quête, voici un indice!"

- 3 Modifiez les messages dans votre programme de la manière décrite précédemment. Vérifiez que vous avez bien modifié chacun de ces messages avant de continuer.

Si vous n'avez pas utilisé de fonctions, c'était probablement un peu fatigant pour un aussi petit changement dans un petit script (à moins que vous maîtrisiez déjà certains raccourcis clavier!). Et pourtant, lorsqu'une équipe développe un logiciel, celui-ci contient bien plus de fichiers et de lignes de code, et il subit de plus grands changements relativement fréquemment.

1.4 Factorisation

En informatique, on parle de factorisation ("refactoring" en anglais) lorsque l'on retravaille le code source d'un programme dans le but d'en améliorer la lisibilité et la maintenance. Lorsqu'un code n'est pas propre, on perd du temps à le maintenir, comme vous venez d'en faire l'expérience. Du coup, vous allez revoir votre code, en faisant bien attention cette fois.

Pour cela, vous allez également documenter vos fonctions. En Python, comme vous l'avez appris lors du cours théorique, la documentation des fonctions se fait via des docstrings. Il existe plusieurs conventions pour rédiger les docstrings, mais ici vous utiliserez une version légèrement modifiée de celle que l'on trouve couramment sur Google :

```
def ma_fonction(parametre1, parametre2):  
    """  
    Une description brève de la fonction.  
  
    Arguments:  
        parametre1 : une description de parametre1  
        parametre2 : une description de parametre2
```

```

Returns:
    Une description de ce que retourne la fonction, si
    elle retourne quelque chose.
"""
<corps de la fonction>

```

Si vous aviez déjà tout écrit avec des fonctions, alors cette sous-section du contrat devrait être très rapide.

1.4.1 Tentative

Une action répétée dans votre code est de demander un nombre au joueur. C'est la première modification que vous allez effectuer. Vous allez créer la fonction `demander()` qui ne prend pas de paramètres et qui a pour but de demander un nombre à l'utilisateur. Elle affiche un message avant chaque tentative, prend l'entrée de l'utilisateur et retourne la valeur entrée en veillant à ce que ce soit un naturel. Dans le cas où l'utilisateur entre autre chose qu'un naturel, la fonction retourne `-1`. Vous pouvez utiliser la méthode `isdigit()` pour vérifier si une chaîne de caractères ne contient que des chiffres et la fonction `int()` pour convertir une chaîne en entier.

4 Écrivez la docstring correspondant à la fonction `demander()`.

5 Écrivez la fonction `demander()`. Refactorisez votre code pour qu'il utilise cette nouvelle fonction. Vérifiez que votre programme fonctionne toujours correctement.

1.4.2 Les indices contre-attaquent

Avant de pouvoir généraliser chaque étape du jeu, il va falloir que vous créiez une fonction pour chaque indice. Chaque indice doit être représenté par une fonction `hintX(generated)` où `X` correspond au numéro de l'indice. Cette fonction prend en entrée un naturel `generated` qui correspond au nombre à deviner. Cette fonction affiche simplement l'indice à l'écran, après avoir vérifié la valeur du nombre à deviner (notez donc que cette fonction ne renvoie rien!).

6 Écrivez la docstring correspondant à la fonction `hintX(generated)` pour chaque indice.

7 Écrivez la fonction `hintX(generated)` pour chaque indice. Refactorisez votre code pour qu'il utilise ces nouvelles fonctions. Vérifiez que votre programme fonctionne toujours correctement.

1.4.3 Dernière étape

Vous pouvez maintenant créer une fonction représentant une étape de manière générale. Cette fonction `step(generated, hint_function)` prend en entrée un naturel `generated` qui correspond au nombre à deviner, et une fonction `hint_function` correspondant à la fonction à appeler pour donner l'indice.

Cette fonction commence par afficher l'indice correspondant à l'étape. Pour ce faire, la fonction passée en paramètre peut simplement être appelée de la même manière qu'une fonction normale (on écrira donc `hint_function(generated)` dans le code de `step`).

Ensuite, cette fonction fait appel à la fonction `demander()` pour récupérer le naturel entré. Finalement, elle vérifie si l'utilisateur a deviné le bon nombre. En fonction de cela, la méthode affiche un message de félicitations ou d'encouragement (notez donc qu'encore une fois cette fonction ne renvoie rien!).

8 Écrivez la docstring correspondant à la fonction `step(generated, hint_function)`.

9 Écrivez la fonction `step(generated, hint_function)`. Refactorisez votre code pour qu'il utilise cette nouvelle fonction. Vérifiez que votre programme fonctionne toujours correctement.

1.4.4 Prêt pour la chemise

Vous avez bien réfléchi, et finalement vous avez encore changé d'avis. Vous décidez de retourner aux messages que vous aviez choisis de base.

10 Modifiez les messages dans votre programme pour retrouver ceux d'avant l'exercice 3.

Vous remarquerez que ces changements sont cette fois beaucoup plus simples que la dernière fois. En effet, cela fait 3 messages à modifier au lieu de 15.

2 Exercices complémentaires

Exam	Date d'une question demandée lors d'un examen ou d'un test (disponible sur moodle avec ou sans corrigé).
★☆☆	Exercice complémentaire – relativement simple ou direct – dont le but est d'aider l'étudiant ayant des difficultés à remplir le contrat de cette série. Il permet par exemple, si nécessaire, de revoir certaines notions de manière plus progressive, avant de s'attaquer au contrat en lui-même.
★★☆	Exercice complémentaire dont le niveau est proche de celui du contrat.
★★★	Exercice complémentaire constituant un challenge plus important ou incluant des subtilités.

★☆☆ 11 Laissez l'utilisateur choisir dans quelle plage de nombre il souhaite pouvoir deviner (par exemple, il pourra choisir entre 10 et 500 s'il le souhaite).

★☆☆ 12 Rajoutez quelques indices pour aider encore plus l'utilisateur.

★★☆ 13 Choisissez le nombre d'essai (et donc d'indices à donner) selon la plage que l'utilisateur vous donne. (Par exemple, vous pouvez donner $n/50$ indices où n est le nombre de numéro possible que l'on peut choisir).

★★☆ 14 Si vous ne l'aviez pas fait avant, refactorisez les points précédents afin d'utiliser des fonctions (par exemple, pour calculer le nombre d'essai).